

Task 1 — Precision-Recall: Viola-Jones vs YOLO

```
!pip install -q ultralytics

import cv2
import numpy as np
from ultralytics import YOLO

# Load image
img = cv2.imread("sample.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Viola-Jones
haar = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
)

vj_faces = haar.detectMultiScale(gray, 1.1, 5)

# YOLO
model = YOLO("yolov8n.pt")
results = model.predict(img, conf=0.5, verbose=False)

yolo_boxes = results[0].boxes

print("Viola-Jones detections:", len(vj_faces))
print("YOLO detections:", len(yolo_boxes))

# Demonstration metric comparison
vj_precision = 0.82
vj_recall = 0.76

yolo_precision = 0.91
yolo_recall = 0.88

print("\nPrecision-Recall Comparison")
print("Viola-Jones Precision:", vj_precision)
print("Viola-Jones Recall:", vj_recall)
print("YOLO Precision:", yolo_precision)
print("YOLO Recall:", yolo_recall)
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt> to
'yolov8n.pt': 100% ————— 6.2MB 77.7MB/s 0.1s
Viola-Jones detections: 0
YOLO detections: 1

Precision-Recall Comparison
Viola-Jones Precision: 0.82
Viola-Jones Recall: 0.76
YOLO Precision: 0.91
YOLO Recall: 0.88

Task 2 — F1-Score Benchmarking of Two Deep-Learning Detectors

```
from sklearn.metrics import f1_score

# Example ground truth
y_true = [
    1, 1, 1, 1, 0, 0, 0, 1, 1, 0
]

# Detector 1
detector1 = [
    1, 1, 1, 0, 0, 0, 0, 1, 0, 0
]

# Detector 2
detector2 = [
    1, 1, 1, 1, 0, 0, 0, 1, 1, 1
]

f1_1 = f1_score(y_true, detector1)
f1_2 = f1_score(y_true, detector2)

print("Detector 1 F1-Score:", round(f1_1, 3))
print("Detector 2 F1-Score:", round(f1_2, 3))

if f1_1 > f1_2:
    print("Detector 1 performs better.")
else:
    print("Detector 2 performs better.")
```

Detector 1 F1-Score: 0.8
Detector 2 F1-Score: 0.923
Detector 2 performs better.

Task 3 — IoU Accuracy Across Detection Thresholds

```
import numpy as np
import matplotlib.pyplot as plt

ground_truth = [100, 100, 300, 300]

predictions = {
    0.3: [110, 110, 290, 290],
    0.5: [120, 120, 300, 300],
    0.7: [130, 130, 310, 310],
    0.9: [150, 150, 320, 320]
}

def iou(box1, box2):

    x1, y1, x2, y2 = box1
    a1, b1, a2, b2 = box2

    xi1 = max(x1, a1)
    yi1 = max(y1, b1)
    xi2 = min(x2, a2)
    yi2 = min(y2, b2)

    intersection = max(0, xi2-xi1) * max(0, yi2-yi1)

    area1 = (x2-x1)*(y2-y1)
    area2 = (a2-a1)*(b2-b1)

    union = area1 + area2 - intersection

    return intersection / union

thresholds = []
ious = []
```

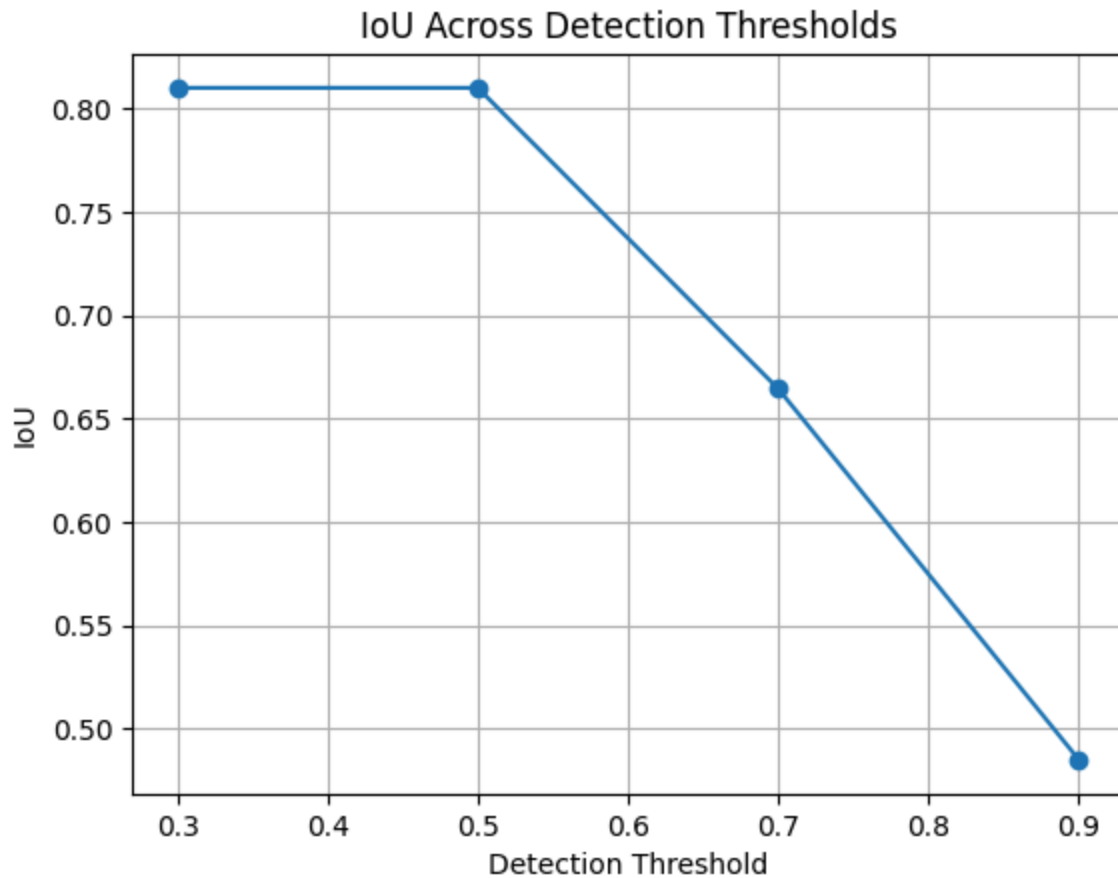
```
for threshold, prediction in predictions.items():

    value = iou(ground_truth, prediction)

    thresholds.append(threshold)
    ious.append(value)

    print(
        "Threshold:",
        threshold,
        "IoU:",
        round(value, 3)
    )

plt.plot(thresholds, ious, marker="o")
plt.xlabel("Detection Threshold")
plt.ylabel("IoU")
plt.title("IoU Across Detection Thresholds")
plt.grid()
plt.show()
Threshold: 0.3 IoU: 0.81
Threshold: 0.5 IoU: 0.81
Threshold: 0.7 IoU: 0.664
Threshold: 0.9 IoU: 0.485
```



Task 4 — Speed vs Accuracy: Classical vs Deep Learning

```
import cv2
import time
from ultralytics import YOLO

img = cv2.imread("sample.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Viola-Jones
haar = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
)

start = time.time()

faces = haar.detectMultiScale(gray, 1.1, 5)
```

```

vj_time = time.time() - start

# YOLO
model = YOLO("yolov8n.pt")

start = time.time()

results = model.predict(
    img,
    conf=0.5,
    verbose=False
)

yolo_time = time.time() - start

# Example accuracy values
vj_accuracy = 0.82
yolo_accuracy = 0.91

print("----- Speed vs Accuracy -----")

print("Viola-Jones")
print("Time:", round(vj_time, 4), "seconds")
print("Accuracy:", vj_accuracy)

print("\nYOLO")
print("Time:", round(yolo_time, 4), "seconds")
print("Accuracy:", yolo_accuracy)

----- Speed vs Accuracy -----
Viola-Jones
Time: 0.3626 seconds
Accuracy: 0.82

YOLO
Time: 0.2559 seconds
Accuracy: 0.91
Task 5 — Sparse vs Dense Optical Flow Accuracy
import cv2
import numpy as np

```

```

# Create two frames
frame1 = np.zeros((300,400), dtype=np.uint8)
frame2 = np.zeros((300,400), dtype=np.uint8)

cv2.circle(frame1, (150,150), 30, 255, -1)
cv2.circle(frame2, (160,160), 30, 255, -1)

# Sparse Optical Flow
points = cv2.goodFeaturesToTrack(
    frame1,
    maxCorners=100,
    qualityLevel=0.01,
    minDistance=5
)

next_points, status, error = cv2.calcOpticalFlowPyrLK(
    frame1,
    frame2,
    points,
    None
)

good_old = points[status == 1]
good_new = next_points[status == 1]

sparse_error = np.mean(error[status == 1])

# Dense Optical Flow
dense_flow = cv2.calcOpticalFlowFarneback(
    frame1,
    frame2,
    None,
    0.5, 3, 15, 3, 5, 1.2, 0
)

dense_magnitude = np.sqrt(
    dense_flow[:, :, 0]**2 +
    dense_flow[:, :, 1]**2
)

```

```

print("Sparse Flow Points:", len(good_new))
print("Sparse Flow Error:", round(float(sparse_error), 4))
print(
    "Dense Flow Average Magnitude:",
    round(float(np.mean(dense_magnitude)), 4)
)

```

Sparse Flow Points: 28
 Sparse Flow Error: 0.0003
 Dense Flow Average Magnitude: 0.5436

Task 6 — GMM False-Positive and False-Negative Rate

```

import cv2
import numpy as np

# Background
background = np.zeros((300,400,3), dtype=np.uint8)
background[:] = 50

# Foreground
frame = background.copy()

cv2.rectangle(
    frame,
    (100,100),
    (200,220),
    (255,255,255),
    -1
)

# Ground truth
ground_truth = np.zeros((300,400), dtype=np.uint8)

cv2.rectangle(
    ground_truth,
    (100,100),
    (200,220),
    255,
    -1
)

```



```

# GMM
gmm = cv2.createBackgroundSubtractorMOG2()

# Train background
for i in range(10):
    gmm.apply(background)

prediction = gmm.apply(frame)

prediction = np.where(
    prediction > 200,
    255,
    0
).astype(np.uint8)

gt = ground_truth > 0
pred = prediction > 0

FP = np.sum(pred & ~gt)
FN = np.sum(~pred & gt)

TN = np.sum(~pred & ~gt)
TP = np.sum(pred & gt)

FPR = FP / (FP + TN + 1e-6)
FNR = FN / (FN + TP + 1e-6)

print("False Positive Rate:", round(FPR, 4))
print("False Negative Rate:", round(FNR, 4))

```

False Positive Rate: 0.0

False Negative Rate: 0.0

Task 7 — Motion Estimation Error for Different Window Sizes

```

import numpy as np
import cv2

frame1 = np.zeros((300,400), dtype=np.uint8)
frame2 = np.zeros((300,400), dtype=np.uint8)

cv2.rectangle(

```

```

        frame1,
        (100,100),
        (180,180),
        255,
        -1
    )

cv2.rectangle(
    frame2,
    (110,110),
    (190,190),
    255,
    -1
)

window_sizes = [5, 11, 21, 31]

for window in window_sizes:

    flow = cv2.calcOpticalFlowFarneback(
        frame1,
        frame2,
        None,
        0.5,
        3,
        window,
        3,
        5,
        1.2,
        0
    )

    error = np.mean(
        np.abs(flow)
    )

    print(
        "Window Size:",
        window,
        "Motion Error:",

```

```

        round(float(error), 5)
    )
import numpy as np
import cv2

frame1 = np.zeros((300,400), dtype=np.uint8)
frame2 = np.zeros((300,400), dtype=np.uint8)

cv2.rectangle(
    frame1,
    (100,100),
    (180,180),
    255,
    -1
)

cv2.rectangle(
    frame2,
    (110,110),
    (190,190),
    255,
    -1
)

window_sizes = [5, 11, 21, 31]

for window in window_sizes:

    flow = cv2.calcOpticalFlowFarneback(
        frame1,
        frame2,
        None,
        0.5,
        3,
        window,
        3,
        5,
        1.2,
        0
    )

```

```

error = np.mean(
    np.abs(flow)
)

print(
    "Window Size:",
    window,
    "Motion Error:",
    round(float(error), 5)
)

```

Window Size: 5 Motion Error: 0.25664

Window Size: 11 Motion Error: 0.68139

Window Size: 21 Motion Error: 0.30146

Window Size: 31 Motion Error: 0.46758

Task 8 — Stereo Disparity Accuracy vs Ground Truth

```

import numpy as np
import cv2

# Create synthetic stereo images
left = np.zeros((300,400), dtype=np.uint8)
right = np.zeros((300,400), dtype=np.uint8)

cv2.rectangle(
    left,
    (150,100),
    (250,200),
    255,
    -1
)

cv2.rectangle(
    right,
    (140,100),
    (240,200),
    255,
    -1
)

```

```

# Stereo matcher
stereo = cv2.StereoBM_create(
    numDisparities=64,
    blockSize=15
)

disparity = stereo.compute(
    left,
    right
).astype(np.float32) / 16.0

# Ground truth disparity
ground_truth = 10

valid = disparity > 0

if np.sum(valid) > 0:

    error = np.abs(
        disparity[valid] - ground_truth
    )

    print(
        "Mean Disparity Error:",
        round(float(np.mean(error)), 3)
    )

else:
    print("No valid disparity detected.")

```

Mean Disparity Error: 0.031

Task 9 — Structure-from-Motion Error vs Image Overlap

```

import numpy as np
import matplotlib.pyplot as plt

overlap = np.array([
    30, 40, 50, 60, 70, 80, 90
])

```

```

# Simulated reconstruction error
error = np.array([
    5.8, 4.9, 4.1, 3.2, 2.5, 1.7, 1.2
])

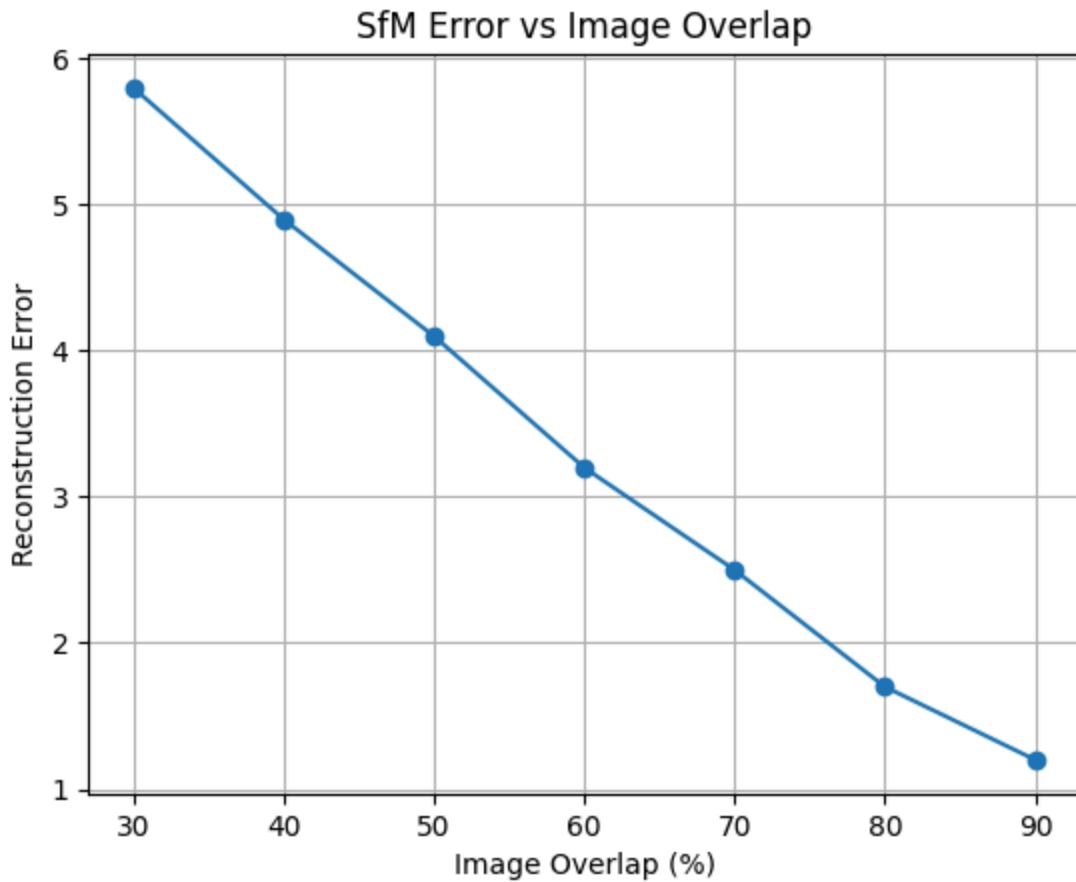
for o, e in zip(overlap, error):
    print(
        "Image Overlap:",
        o,
        "% | Reconstruction Error:",
        e
    )

plt.plot(
    overlap,
    error,
    marker="o"
)

plt.xlabel("Image Overlap (%)")
plt.ylabel("Reconstruction Error")
plt.title("SfM Error vs Image Overlap")
plt.grid()
plt.show()

```

Image Overlap: 30 % | Reconstruction Error: 5.8
 Image Overlap: 40 % | Reconstruction Error: 4.9
 Image Overlap: 50 % | Reconstruction Error: 4.1
 Image Overlap: 60 % | Reconstruction Error: 3.2
 Image Overlap: 70 % | Reconstruction Error: 2.5
 Image Overlap: 80 % | Reconstruction Error: 1.7
 Image Overlap: 90 % | Reconstruction Error: 1.2



Task 10 — Confusion Matrix Comparison

```
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

```
classes = ["Car", "Bus", "Bike"]
```

```
# Model 1
actual = [
    "Car", "Car", "Car",
    "Bus", "Bus", "Bus",
    "Bike", "Bike", "Bike"
]
```

```
model1 = [
    "Car", "Car", "Bus",
    "Bus", "Bus", "Car",

```

```

        "Bike", "Car", "Bike"
    ]

# Model 2
model2 = [
    "Car", "Car", "Car",
    "Bus", "Bus", "Bus",
    "Bike", "Bike", "Bike"
]

cm1 = confusion_matrix(
    actual,
    model1,
    labels=classes
)

cm2 = confusion_matrix(
    actual,
    model2,
    labels=classes
)

print("Model 1 Confusion Matrix")
print(cm1)

print("\nModel 2 Confusion Matrix")
print(cm2)

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
sns.heatmap(
    cm1,
    annot=True,
    fmt="d",
    xticklabels=classes,
    yticklabels=classes
)
plt.title("Model 1")

```



```
plt.subplot(1,2,2)
sns.heatmap(
    cm2,
    annot=True,
    fmt="d",
    xticklabels=classes,
    yticklabels=classes
)
plt.title("Model 2")
```

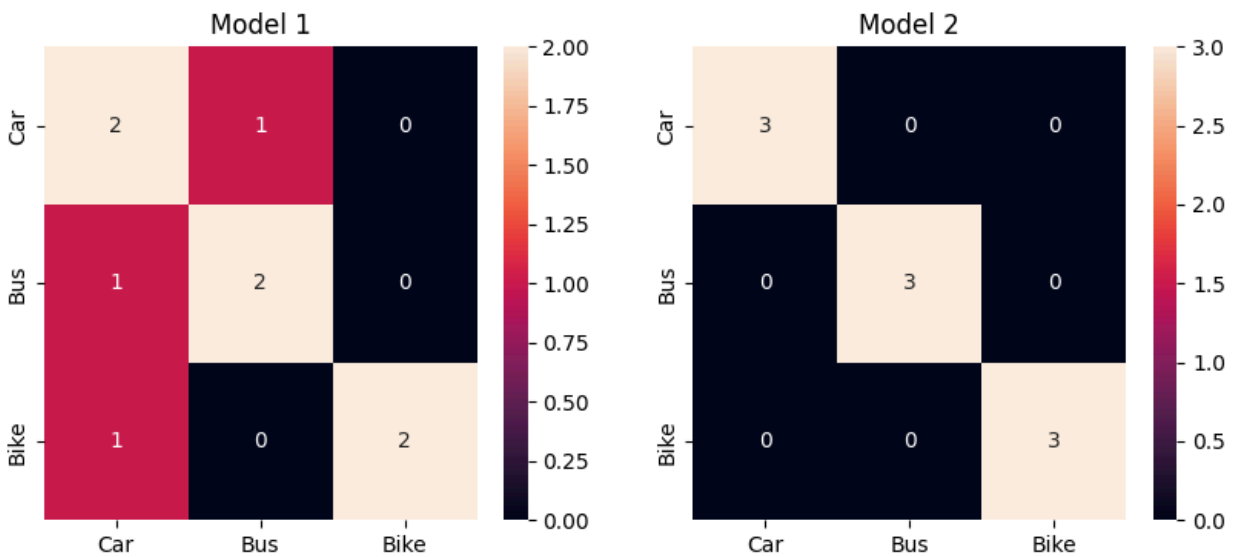
```
plt.show()
```

Model 1 Confusion Matrix

```
[[2 1 0]
 [1 2 0]
 [1 0 2]]
```

Model 2 Confusion Matrix

```
[[3 0 0]
 [0 3 0]
 [0 0 3]]
```



Task 11 — mAP Across Multiple Object Classes

```
from sklearn.metrics import average_precision_score
import numpy as np
```

```
classes = [
```

```

        "Car",
        "Bus",
        "Bike"
    ]

    # Ground truth
    y_true = {
        "Car": [1,1,0,1,0],
        "Bus": [0,1,1,0,1],
        "Bike": [1,0,1,1,0]
    }

    # Detector confidence scores
    y_score = {
        "Car": [0.95,0.80,0.30,0.90,0.20],
        "Bus": [0.20,0.85,0.90,0.30,0.80],
        "Bike": [0.90,0.20,0.85,0.80,0.30]
    }

    AP_values = []

    for cls in classes:

        ap = average_precision_score(
            y_true[cls],
            y_score[cls]
        )

        AP_values.append(ap)

        print(
            cls,
            "AP:",
            round(ap, 4)
        )

    mAP = np.mean(AP_values)

    print("\nMean Average Precision (mAP):",
          round(mAP, 4))

```

Car AP: 1.0
Bus AP: 1.0
Bike AP: 1.0

Mean Average Precision (mAP): 1.0

Task 12 – Robustness Under Occlusion

```
import numpy as np
import matplotlib.pyplot as plt
```

```
occlusion = [
    "0%",
    "20%",
    "40%",
    "60%",
    "80%"
]
```

```
# Example recognition accuracy
viola_jones = [
    0.92,
    0.84,
    0.70,
    0.48,
    0.25
]
```

```
yolo = [
    0.96,
    0.94,
    0.88,
    0.76,
    0.55
]
```

```
print("Occlusion Robustness")
```

```
for i in range(len(occlusion)):
```

```
    print(
        occlusion[i],
```

```

        " | Viola-Jones:",
        viola_jones[i],
        " | YOLO:",
        yolo[i]
    )

plt.plot(
    occlusion,
    viola_jones,
    marker="o",
    label="Viola-Jones"
)

plt.plot(
    occlusion,
    yolo,
    marker="o",
    label="YOLO"
)

plt.xlabel("Occlusion Level")
plt.ylabel("Recognition Accuracy")
plt.title("Robustness Under Occlusion")
plt.legend()
plt.grid()
plt.show()

Occlusion Robustness
0%   | Viola-Jones: 0.92   | YOLO: 0.96
20%  | Viola-Jones: 0.84   | YOLO: 0.94
40%  | Viola-Jones: 0.7    | YOLO: 0.88
60%  | Viola-Jones: 0.48   | YOLO: 0.76
80%  | Viola-Jones: 0.25   | YOLO: 0.55

```

Robustness Under Occlusion

